Open in Colab

T81-558: Applications of Deep Neural Networks

Module 13: Advanced/Other Topics

- Instructor: Jeff Heaton, McKelvey School of Engineering, Washington University in St. Louis
- For more information visit the class website.

Module 13 Video Material

- **Part 13.1: Flask and Deep Learning Web Services** [Video] [Notebook]
- Part 13.2: Interrupting and Continuing Training [Video] [Notebook]
- Part 13.3: Using a Keras Deep Neural Network with a Web Application [Video] [Notebook]
- Part 13.4: When to Retrain Your Neural Network [Video] [Notebook]
- Part 13.5: Tensor Processing Units (TPUs) [Video] [Notebook]

Part 13.1: Flask and Deep Learning Web Services

Suppose you would like to create websites based on neural networks. In that case, we must expose the neural network so that Python and other programming languages can efficiently execute. The usual means for such integration is a web service. One of the most popular libraries for doing this in Python is Flask. This library allows you to quickly deploy your Python applications, including TensorFlow, as web services.

Neural network deployment is a complex process, usually carried out by a company's Information Technology (IT) group. When large numbers of clients must access your model, scalability becomes essential. The cloud usually handles this. The designers of Flask did not design for high-volume systems. When deployed to production, you will wrap models in Gunicorn or TensorFlow Serving. We will discuss high-volume cloud deployment in the next section. Everything presented in this part ith Flask is directly compatible with the higher volume Gunicorn system. When early in the development process, it is common to use Flask directly.

Flask Hello World

Flask is the server, and Jupyter usually fills the role of the client. It is uncommon to run Flask from a Jupyter notebook. However, we can run a simple web service from Jupyter. We will quickly move beyond this and deploy using a Python script (.py). Because we must use .py files, it won't be easy to use Google CoLab, as you will be running from the command line. For now, let's execute a Flask web container in Jupyter.

In [1]:
```
from werkzeug.wrappers import Request, Response
from flask import Flask

app = Flask(__name__)

@app.route("/")
def hello():
    return "Hello World!"

if __name__ == '__main__':
    from werkzeug.serving import run_simple
    run_simple('localhost', 9000, app)
```

```
 * Running on http://localhost:9000/ (Press CTRL+C to quit)
127.0.0.1 - - [01/Aug/2019 14:51:25] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [01/Aug/2019 14:51:25] "GET /favicon.ico HTTP/1.1" 404 -
```

This program starts a web service on port 9000 of your computer. This cell will remain running (appearing locked up). However, it is merely waiting for browsers to connect. If you point your browser at the following URL, you will interact with the Flask web service.

- http://localhost:9000/

You should see Hello World displayed.

MPG Flask

Usually, you will interact with a web service through JSON. A program will send a JSON message to your Flask application, and your Flask application will return a JSON. Later, in module 13.3, we will see how to attach this web service to a web application that you can interact with through a browser. We will create a Flask wrapper for a neural network that predicts the miles per gallon. The sample JSON will look like this.

```
{
  "cylinders": 8,
  "displacement": 300,
  "horsepower": 78,
  "weight": 3500,
  "acceleration": 20,
  "year": 76,
```

```
    "origin": 1
  }
```

We will see two different means of POSTing this JSON data to our web server. First, we will use a utility called POSTman. Secondly, we will use Python code to construct the JSON message and interact with Flask.

First, it is necessary to train a neural network with the MPG dataset. This technique is very similar to what we've done many times before. However, we will save the neural network so that we can load it later. We do not want to have Flask train the neural network. We wish to have the neural network already trained and deploy the already prepared .H5 file to save the neural network. The following code trains an MPG neural network.

In [2]:
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Activation
from sklearn.model_selection import train_test_split
from tensorflow.keras.callbacks import EarlyStopping
import pandas as pd
import io
import os
import requests
import numpy as np
from sklearn import metrics

df = pd.read_csv(
    "https://data.heatonresearch.com/data/t81-558/auto-mpg.csv",
    na_values=['NA', '?'])

cars = df['name']

# Handle missing value
df['horsepower'] = df['horsepower'].fillna(df['horsepower'].median())

# Pandas to Numpy
x = df[['cylinders', 'displacement', 'horsepower', 'weight',
       'acceleration', 'year', 'origin']].values
y = df['mpg'].values # regression

# Split into validation and training sets
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.25, random_state=42)

# Build the neural network
model = Sequential()
model.add(Dense(25, input_dim=x.shape[1], activation='relu')) # Hidden 1
model.add(Dense(10, activation='relu')) # Hidden 2
model.add(Dense(1)) # Output
model.compile(loss='mean_squared_error', optimizer='adam')

monitor = EarlyStopping(monitor='val_loss', min_delta=1e-3, patience=5, \
        verbose=1, mode='auto',\
        restore_best_weights=True)
```

```
model.fit(x_train,y_train,validation_data=(x_test,y_test),\
          callbacks=[monitor],verbose=2,epochs=1000)
```

```
WARNING: Logging before flag parsing goes to stderr.
W0801 14:54:30.408809 140735710749568 deprecation.py:323] From /Users/jheaton/minico
nda3/envs/tensorflow/lib/python3.6/site-packages/tensorflow_core/python/keras/optimi
zer_v2/optimizer_v2.py:468: BaseResourceVariable.constraint (from tensorflow.python.
ops.resource_variable_ops) is deprecated and will be removed in a future version.
Instructions for updating:
Apply a constraint manually following the optimizer update step.
```

```
Train on 298 samples, validate on 100 samples
Epoch 1/1000
298/298 - 0s - loss: 2009.9526 - val_loss: 1452.2106
Epoch 2/1000
298/298 - 0s - loss: 562.4718 - val_loss: 532.2283
Epoch 3/1000
298/298 - 0s - loss: 331.3064 - val_loss: 229.2169
Epoch 4/1000
298/298 - 0s - loss: 197.1110 - val_loss: 149.5195
Epoch 5/1000
298/298 - 0s - loss: 167.2275 - val_loss: 137.9593
Epoch 6/1000
298/298 - 0s - loss: 141.5222 - val_loss: 123.4302
Epoch 7/1000
298/298 - 0s - loss: 134.0808 - val_loss: 119.0394
Epoch 8/1000
298/298 - 0s - loss: 128.3394 - val_loss: 116.1934
Epoch 9/1000
298/298 - 0s - loss: 125.9083 - val_loss: 111.8186
Epoch 10/1000
298/298 - 0s - loss: 122.4650 - val_loss: 108.0787
Epoch 11/1000
298/298 - 0s - loss: 123.8484 - val_loss: 117.2898
Epoch 12/1000
298/298 - 0s - loss: 122.1327 - val_loss: 102.7727
Epoch 13/1000
298/298 - 0s - loss: 114.5446 - val_loss: 97.1289
Epoch 14/1000
298/298 - 0s - loss: 109.9774 - val_loss: 104.2486
Epoch 15/1000
298/298 - 0s - loss: 103.9899 - val_loss: 97.3281
Epoch 16/1000
298/298 - 0s - loss: 106.6320 - val_loss: 94.8660
Epoch 17/1000
298/298 - 0s - loss: 99.9431 - val_loss: 87.0038
Epoch 18/1000
298/298 - 0s - loss: 99.5071 - val_loss: 84.3153
Epoch 19/1000
298/298 - 0s - loss: 97.8698 - val_loss: 76.5715
Epoch 20/1000
298/298 - 0s - loss: 89.4999 - val_loss: 85.0211
Epoch 21/1000
298/298 - 0s - loss: 93.6648 - val_loss: 77.6890
Epoch 22/1000
298/298 - 0s - loss: 84.4505 - val_loss: 67.5825
Epoch 23/1000
298/298 - 0s - loss: 80.5943 - val_loss: 65.0318
Epoch 24/1000
298/298 - 0s - loss: 84.3871 - val_loss: 71.8327
Epoch 25/1000
298/298 - 0s - loss: 73.9898 - val_loss: 59.8741
Epoch 26/1000
298/298 - 0s - loss: 72.3061 - val_loss: 64.4955
Epoch 27/1000
298/298 - 0s - loss: 71.3641 - val_loss: 60.4624
Epoch 28/1000
```

```
298/298 - 0s - loss: 68.7283 - val_loss: 53.2123
Epoch 29/1000
298/298 - 0s - loss: 65.7615 - val_loss: 53.1017
Epoch 30/1000
298/298 - 0s - loss: 64.4437 - val_loss: 50.6363
Epoch 31/1000
298/298 - 0s - loss: 61.7922 - val_loss: 53.4028
Epoch 32/1000
298/298 - 0s - loss: 60.4436 - val_loss: 45.5426
Epoch 33/1000
298/298 - 0s - loss: 58.2074 - val_loss: 44.6183
Epoch 34/1000
298/298 - 0s - loss: 58.3198 - val_loss: 42.4181
Epoch 35/1000
298/298 - 0s - loss: 55.2720 - val_loss: 41.4758
Epoch 36/1000
298/298 - 0s - loss: 54.0140 - val_loss: 39.8373
Epoch 37/1000
298/298 - 0s - loss: 52.7805 - val_loss: 50.7949
Epoch 38/1000
298/298 - 0s - loss: 52.7188 - val_loss: 38.0612
Epoch 39/1000
298/298 - 0s - loss: 49.0947 - val_loss: 37.0849
Epoch 40/1000
298/298 - 0s - loss: 48.5237 - val_loss: 35.5431
Epoch 41/1000
298/298 - 0s - loss: 47.1861 - val_loss: 34.2021
Epoch 42/1000
298/298 - 0s - loss: 45.6822 - val_loss: 37.6906
Epoch 43/1000
298/298 - 0s - loss: 44.9434 - val_loss: 34.6666
Epoch 44/1000
298/298 - 0s - loss: 43.5861 - val_loss: 32.9166
Epoch 45/1000
298/298 - 0s - loss: 43.2145 - val_loss: 39.4031
Epoch 46/1000
298/298 - 0s - loss: 42.4890 - val_loss: 30.3377
Epoch 47/1000
298/298 - 0s - loss: 41.9875 - val_loss: 29.8683
Epoch 48/1000
298/298 - 0s - loss: 40.9617 - val_loss: 30.6645
Epoch 49/1000
298/298 - 0s - loss: 40.4144 - val_loss: 34.1248
Epoch 50/1000
298/298 - 0s - loss: 40.4761 - val_loss: 33.8365
Epoch 51/1000
298/298 - 0s - loss: 39.0555 - val_loss: 31.4981
Epoch 52/1000
Restoring model weights from the end of the best epoch.
298/298 - 0s - loss: 37.9472 - val_loss: 32.6139
Epoch 00052: early stopping
```

Out[2]:

```
<tensorflow.python.keras.callbacks.History at 0x12df96208>
```

Next, we evaluate the score. This evaluation is more of a sanity check to ensure the code above worked as expected.

In [3]:
```
pred = model.predict(x_test)
# Measure RMSE error.  RMSE is common for regression.
score = np.sqrt(metrics.mean_squared_error(pred,y_test))
print(f"After load score (RMSE): {score}")
```

```
After load score (RMSE): 5.465193688130732
```

Next, we save the neural network to a .H5 file.

In [4]:
```
model.save(os.path.join("./dnn/","mpg_model.h5"))
```

We want the Flask web service to check that the input JSON is valid. To do this, we need to know what values we expect and their logical ranges. The following code outputs the expected fields and their ranges, and packages all of this information into a JSON object that you should copy to the Flask web application. This code allows us to validate the incoming JSON requests.

In [5]:
```
cols = [x for x in df.columns if x not in ('mpg','name')]

print("{")
for i,name in enumerate(cols):
    print(f'"{name}":{{"min":{df[name].min()},\
          "max":{df[name].max()}}}{"," if i<(len(cols)-1) else ""}')
print("}")
```

```
{
"cylinders":{"min":3,"max":8},
"displacement":{"min":68.0,"max":455.0},
"horsepower":{"min":46.0,"max":230.0},
"weight":{"min":1613,"max":5140},
"acceleration":{"min":8.0,"max":24.8},
"year":{"min":70,"max":82},
"origin":{"min":1,"max":3}
}
```

Finally, we set up the Python code to call the model for a single car and get a prediction. You should also copy this code to the Flask web application.

In [6]:
```
import os
from tensorflow.keras.models import load_model
import numpy as np

model = load_model(os.path.join("./dnn/","mpg_model.h5"))
x = np.zeros( (1,7) )

x[0,0] = 8 # 'cylinders',
x[0,1] = 400 # 'displacement',
x[0,2] = 80 # 'horsepower',
x[0,3] = 2000 # 'weight',
x[0,4] = 19 # 'acceleration',
```

```
x[0,5] = 72 # 'year',
x[0,6] = 1 # 'origin'


pred = model.predict(x)
float(pred[0])
```

Out[6]: 6.212100505828857

The completed web application can be found here:

- mpg_server_1.py

You can run this server from the command line with the following command:

```
python mpg_server_1.py
```

If you are using a virtual environment (described in Module 1.1), use the `activate tensorflow` command for Windows or `source activate tensorflow` for Mac before executing the above command.

Flask MPG Client

Now that we have a web service running, we would like to access it. This server is a bit more complicated than the "Hello World" web server we first saw in this part. The request to display was an HTTP GET. We must now do an HTTP POST. To accomplish access to a web service, you must use a client. We will see how to use PostMan and directly through a Python program in Jupyter.

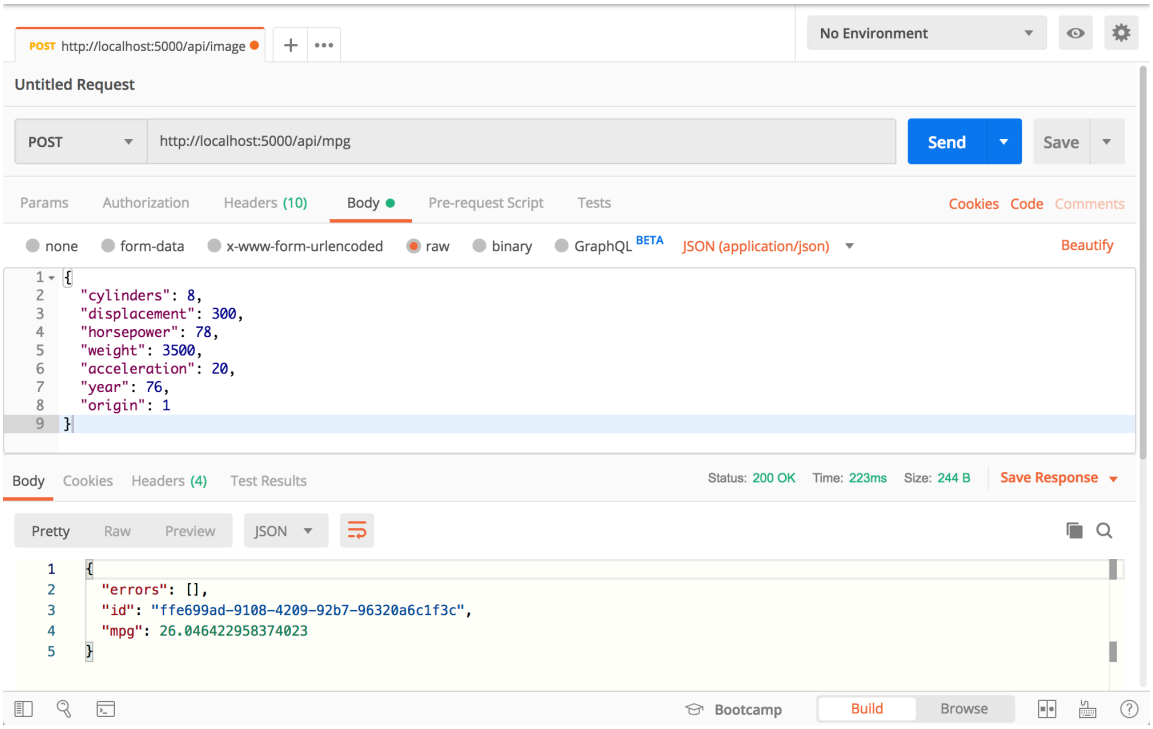
We will begin with PostMan. If you have not already done so, install PostMan.

To successfully use PostMan to query your web service, you must enter the following settings:

- POST Request to http://localhost:5000/api/mpg
- RAW JSON and paste in JSON from above
- Click Send and you should get a correct result

Figure 13.PM shows a successful result.

Figure 13.PM: PostMan JSON



This same process can be done programmatically in Python.

In [7]:
```
import requests

json = {
  "cylinders": 8,
  "displacement": 300,
  "horsepower": 78,
  "weight": 3500,
  "acceleration": 20,
  "year": 76,
  "origin": 1
}

r = requests.post("http://localhost:5000/api/mpg",json=json)
if r.status_code == 200:
    print("Success: {}".format(r.text))
else: print("Failure: {}".format(r.text))
```

```
Success: {
  "errors": [],
  "id": "643d027e-554f-4401-ba5f-78592ae7e070",
  "mpg": 23.885438919067383
}
```

Images and Web Services

We can also accept images from web services. We will create a web service that accepts images and classifies them using MobileNet. You will follow the same process; load

your network as we did for the MPG example. You can find the completed web service can here:

image_server_1.py

You can run this server from the command line with:

```
python mpg_server_1.py
```

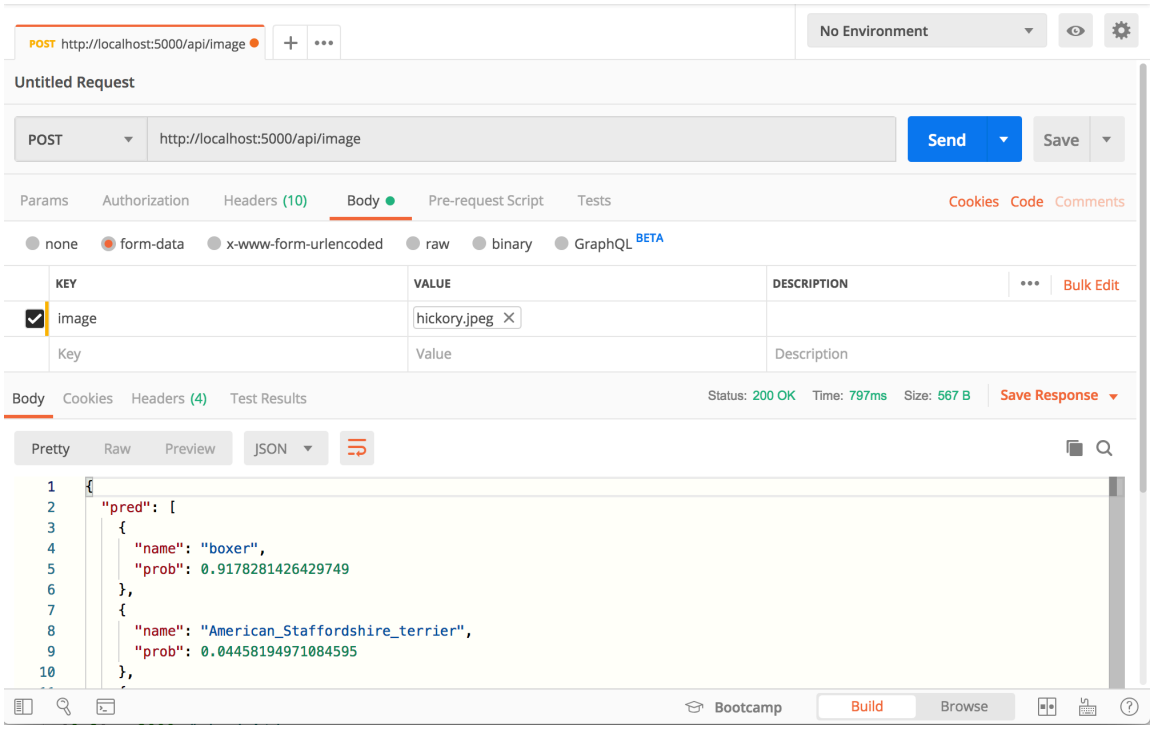
If you are using a virtual environment (described in Module 1.1), use the `activate tensorflow` command for Windows or `source activate tensorflow` for Mac before executing the above command.

To successfully use PostMan to query your web service, you must enter the following settings:

- POST Request to http://localhost:5000/api/image
- Use "Form Data" and create one entry named "image" that is a file. Choose an image file to classify.
- Click Send, and you should get a correct result

Figure 13.PMI shows a successful result.

Figure 13.PMI: PostMan Images



This same process can be done programmatically in Python.

In [8]:
```
import requests
response = requests.post('http://localhost:5000/api/image', files=\
        dict(image=('hickory.jpeg',open('./photos/hickory.jpeg','rb'))))
```

```python
if response.status_code == 200:
    print("Success: {}".format(response.text))
else: print("Failure: {}".format(response.text))
```

```
Success: {
  "pred": [
    {
      "name": "boxer",
      "prob": 0.9178281426429749
    },
    {
      "name": "American_Staffordshire_terrier",
      "prob": 0.04458194971084595
    },
    {
      "name": "French_bulldog",
      "prob": 0.018736232072114944
    },
    {
      "name": "bull_mastiff",
      "prob": 0.016469275578856468
    },
    {
      "name": "pug",
      "prob": 0.0009862519800662994
    }
  ]
}
```